EDA-Practice-2025-07-20

Minsung Kim

2025-09-28

```
# Load applicable libraries
library(dplyr)
library(tidyr)
library(zoo)
library(lubridate)
library(xts)
library(dygraphs)

# Load data
df <- read.csv("concurrent account spending_redacted.csv")
```

Data Cleaning

We will do the following to prepare the data for the time-series plot.

- Combine the Debit and Credit into one column of ‘Values’ and remove the -/+ signs (using absolute value).
- Instead of relying on `SAVING` category, we will create two separate graphs. First graph will illustrate daily spending broken down into `NEEDS`, `WANTS`, and `MOM` (last one standing for money sent to my mom).

Second graph will compare combined spending against `INCOME`. The reason I chose not to use the `SAVING` indicator is that it is unreliable. Unfortunately, I only recorded positive amounts saved for the given month without accounting for “negative” savings (when spending exceeded revenue) during data entry. Therefore, I will aim to create a new dedicated `SAVINGS` column that takes the difference between `SPENDING` and `INCOME` at any given time.

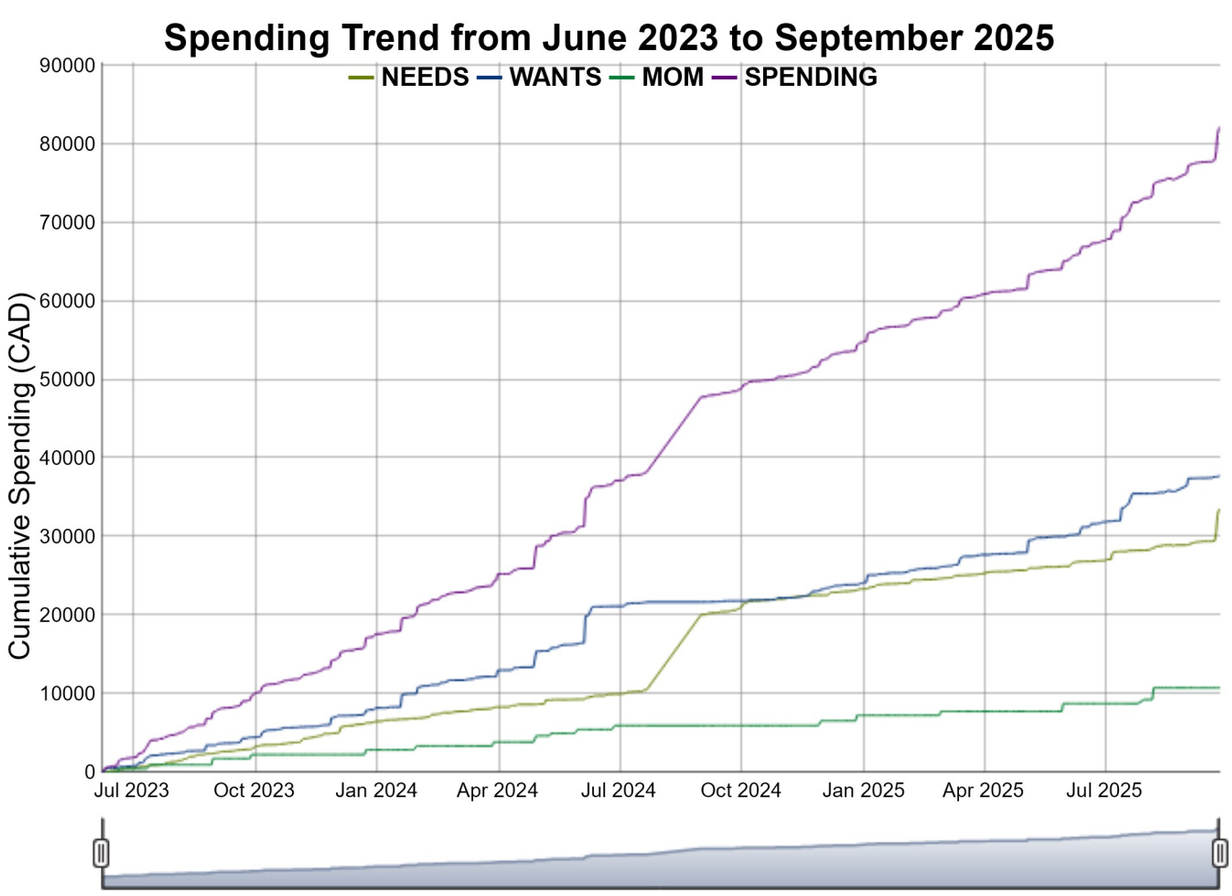
- Combine all duplicate dates so each row represents a unique date and the **cumulative value** of the spending type.
- Rearrange the data so that each column represents spending type (e.g., `NEEDS`, `WANTS`, `MOM`) for each date row instead of having multiple date rows for different spending types.
- Populate missing values (depending on how `dygraphs` handles missing data).
- Convert the data into time-series compatible (xts) format to handle `dygraphs`.

```
# replace "" under category as "INCOME". This will come in handy later when we create another
plot of overall spending vs income
df$Category[df$Category == ""] <- "INCOME"

# replace NA with 0
df <- df %>% mutate_at(vars(Debit, Credit), ~replace(., is.na(.), 0))

# combine the debit and credit columns into one column of absolute values called 'value'
df <- df %>% mutate(value = abs(Debit + Credit))

# create a new column called 'cum_sum' which takes the values from the `value` column and obt
ains a cumulative sum. Afterwards, select only the `last` date row if there are multiple iden
tical date rows for a given spending type. All other identical date rows (for a given spendin
g type) are removed since the last row shows the final cumulative value.
df_NEED <- df %>% filter(Category == "NEEDS") %>% mutate(cum_value = round(cumsum(value),2))
%>% group_by(Transaction.Date) %>% summarise_all(last)
df_WANT <- df %>% filter(Category == "WANTS") %>% mutate(cum_value = round(cumsum(value),2))
%>% group_by(Transaction.Date) %>% summarise_all(last)
df_MOM <- df %>% filter(Category == "MOM") %>% mutate(cum_value = round(cumsum(value),2)) %>%
group_by(Transaction.Date) %>% summarise_all(last)

# combine the three datasets using row bind and rearrange the trimmed dataset by ascending da
te order. Remove all redundant columns.
df_trimmed <- bind_rows(df_NEED, df_WANT, df_MOM) %>% select(Transaction.Date, Category, Cumu
lative.Value = cum_value) %>% arrange(Transaction.Date)

# rearrange the data so that each spending type is given their own columns per date.
df_trimmed <- df_trimmed %>% pivot_wider(names_from = Category, values_from = Cumulative.Valu
e)

# fill in the NA values by referencing the previous non-NA value and carrying it forward
# set a baseline of 0 for columns in the first row missing values
df_trimmed$WANTS[1] <- 0
df_trimmed$MOM[1] <- 0

# carry the previous value foward for every NA values
df_trimmed$MOM <- na.locf(df_trimmed$MOM)
df_trimmed$NEEDS <- na.locf(df_trimmed$NEEDS)
df_trimmed$WANTS <- na.locf(df_trimmed$WANTS)
```

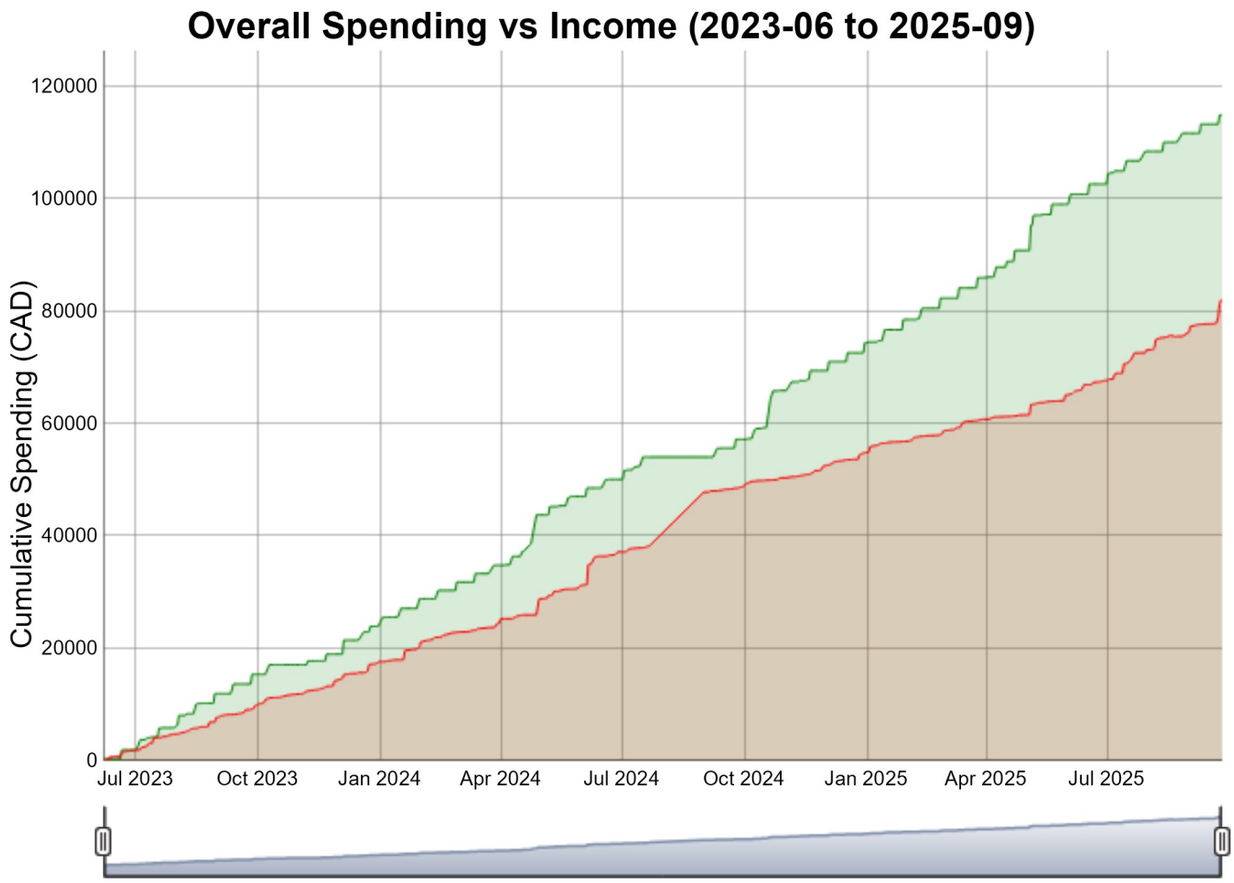
```
# create a separate df containing only `INCOME` data. Trim the columns so only cumulative inc
ome and Transaction date (used as keys for JOIN function later)
df_INCOME <- df %>% filter(Category == "INCOME") %>% mutate(cum_value = round(cumsum(valu
e),2)) %>% group_by(Transaction.Date) %>% summarise_all(last) %>% select(Transaction.Date, cu
m_value)

# create a separate column for all spending (MOM + NEEDS + WANTS)
df_trimmed <- df_trimmed %>% mutate(SPENDING = MOM + NEEDS + WANTS)

# combine the truncated data with the income data for overall spending vs income chart
df_cum.spending <- full_join(select(df_trimmed, Transaction.Date, SPENDING), df_INCOME, by =
join_by(Transaction.Date)) %>%
  rename(INCOME = cum_value) %>% arrange(Transaction.Date)

# repeat the same steps as above to apply a baseline income of 0 and replace all NA values wi
th the previous non-NA observation
df_cum.spending$SPENDING[1] <- 0

# replace all following NA values from each column with the previous non-NA observation using
`na.locf` (last observation carried forward)
df_cum.spending$INCOME <- na.locf(df_cum.spending$INCOME)
df_cum.spending$SPENDING <- na.locf(df_cum.spending$SPENDING)

# convert df into xts time-series format. The original `Transaction.Date` column is removed a
s xts function will automatically created a new date column using the date-converted `df_trim
med$Transaction.Date` following the order.by parameter.
time_series_df <- xts(df_trimmed[,-1], order.by = as_date(df_trimmed$Transaction.Date))
time_series_df_cum.spending <- xts(df_cum.spending[, -1], order.by = as_date(df_cum.spending
$Transaction.Date))
```

Viz

```
# plot interactive plot of spending from 2023 June to 2025 June.
dygraph(time_series_df, main = "Spending Trend from June 2023 to September 2025") %>% dyAxis
("y", label = "Cumulative Spending (CAD)") %>% dyLegend(width = 475) %>% dyRangeSelector()
%>% dyCSS("dygraph.css")
```



```
# plot overall spending vs overall income
dygraph(time_series_df_cum.spending, main = "Overall Spending vs Income (2023-06 to 2025-0
9)") %>% dyAxis("y", label = "Cumulative Spending (CAD)") %>% dyLegend(show = "follow") %>% d
yRangeSelector() %>% dyCSS("dygraph.css") %>% dySeries("INCOME", fillGraph = TRUE, color = "g
reen") %>% dySeries("SPENDING", fillGraph = TRUE, color = "red")
```



Credit

https://rstudio.github.io/dygraphs/gallery-series-highlighting.html (https://rstudio.github.io/dygraphs/gallery-series-highlighting.html)